

Лабораторная работа №2

Структуры данных

Умед Джамshedович Назармамадов

В отчёте рассмотрены основные структуры данных языка Julia: кортежи, словари, множества и массивы, а также общие функции для работы с ними (`isempty`, `length`, `in`, `unique`, `reduce`, `maximum/minimum`).

Воспроизведены примеры из методических указаний и выполнены задания для самостоятельной работы: операции над множествами, различные способы построения массивов и векторов, работа с пакетом `Primes`, вычисление сумм рядов.

Все результаты подтверждены скриншотами выполнения кода в Jupyter Lab.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	8
3	Материалы и методы	9
4	Выполнение лабораторной работы	10
4.1	Подготовка	10
4.2	Раздел 2.2. Структуры данных	11
4.3	Задания для самостоятельной работы (раздел 2.4)	14
5	Результаты	21
6	Обсуждение результатов	22

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	7
4.1	Создание ветки feature/lab02	10
4.2	Создание каталога проекта и запуск Jupyter Lab	10
4.3	Определение кортежей	11
4.4	Операции над кортежами	11
4.5	Операции над словарями	12
4.6	Операции над множествами	12
4.7	Операции над массивами	13
4.8	Задание 1 (множества A, B, C) и задание 2 (множества смешанных типов)	14
4.9	Задание 3, пункты 3.1–3.14 (код)	16
4.10	Задание 3, пункты 3.1–3.14 (продолжение и результат)	17
4.11	Задание 4: массив квадратов чисел	18
4.12	Задание 5: пакет Primes	19
4.13	Задание 6: вычисление сумм	20

Список таблиц

1 Введение

1.1 Актуальность

Структуры данных — кортежи, словари, множества и массивы — базовый инструментарий для представления и обработки данных в любой задаче статистического анализа. Понимание их особенностей в Julia (неизменяемость кортежей, неупорядоченность словарей и множеств, широкие возможности индексации и broadcasting для массивов) необходимо для дальнейшей работы с данными в курсе.

1.2 Цель работы

Изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

1.3 Задачи

1. Используя Jupyter Lab, повторить примеры из раздела 2.2 методических указаний (кортежи, словари, множества, массивы).
2. Выполнить задания для самостоятельной работы (раздел 2.4): операции над множествами, построение массивов различными способами, работа с пакетом `Primes`, вычисление сумм рядов.

1.4 Объект и предмет исследования

Объект исследования — структуры данных языка Julia (кортежи, словари, множества, массивы). Предмет исследования — операции над этими структурами и функции, общие для всех структур данных (`isempty()`, `length()`, `in()`, `unique()`, `reduce()`, `maximum()`/`minimum()`).

1.5 Методы исследования

Практическая работа в интерактивной среде Jupyter Lab: последовательное выполнение примеров кода в ячейках блокнота с анализом полученных результатов.

1.6 Схема выполнения работы

На рис. -Рисунок 1.1 представлена общая последовательность действий при выполнении работы.



Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Кортеж (Tuple) — неизменяемая индексируемая последовательность элементов, индексация с единицы; поддерживает именованные поля (NamedTuple).

Словарь (Dict) — неупорядоченный набор пар «ключ — значение» с доступом по ключу.

Множество (Set) — неупорядоченная совокупность уникальных элементов, поддерживающая теоретико-множественные операции (union, intersect, setdiff, issubset).

Массив — упорядоченная коллекция элементов на многомерной сетке; векторы и матрицы — частные случаи массива. Поддерживает генераторы (list comprehension), broadcasting (. -операции) и гибкую индексацию срезами.

Общие для всех структур функции: isempty(), length(), in(), unique(), reduce(), maximum()/minimum().

3 Материалы и методы

Окружение то же, что и в лабораторной работе №1: WSL2 Ubuntu, Julia 1.10.5, Jupyter Lab 4.6.3 с ядром IJulia. Работа выполнена в ветке `feature/lab02` git-репозитория `2026-2--study--computer-practice`, блокнот `data-structures.ipynb` создан в `labs/lab02/project/`.

4 Выполнение лабораторной работы

4.1 Подготовка

Создана ветка `feature/lab02` (рис. -Рисунок 4.1) и рабочий каталог для блокнота, запущен Jupyter Lab (рис. -Рисунок 4.2).

```
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ git flow feature start lab02
Switched to a new branch 'feature/lab02'

Summary of actions:
- A new branch 'feature/lab02' was created, based on 'develop'
- You are now on branch 'feature/lab02'

Now, start committing on your feature. When done, use:

    git flow feature finish lab02
```

Рисунок 4.1: Создание ветки `feature/lab02`

```
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ mkdir -p labs/lab02/project
umedn@HUAWEI:~/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice$ jupyter lab
[I 2026-09-26 20:38:27.169 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-09-26 20:38:27.172 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-09-26 20:38:27.175 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-09-26 20:38:27.556 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-09-26 20:38:27.624 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-09-26 20:38:27.625 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-09-26 20:38:27.626 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-09-26 20:38:27.631 LabApp] JupyterLab extension loaded from /home/umedn/.local/lib/python3.12/site-packages/jupyterlab
[I 2026-09-26 20:38:27.631 LabApp] JupyterLab application directory is /home/umedn/.local/share/jupyter/lab
[I 2026-09-26 20:38:27.632 LabApp] Extension Manager is 'pypi'.
[I 2026-09-26 20:38:27.685 ServerApp] jupyterlab | extension was successfully loaded.
[I 2026-09-26 20:38:27.686 ServerApp] Serving notebooks from local directory: /home/umedn/work/study/2026-2/2026-2==computer-practice/2026-2--study--computer-practice
```

Рисунок 4.2: Создание каталога проекта и запуск Jupyter Lab

4.2 Раздел 2.2. Структуры данных

4.2.1 Кортежи

Определены пустой кортеж, кортеж строк, кортеж чисел, кортеж со смешанными типами и именованный кортеж (рис. -Рисунок 4.3).

```
[1]: ()
      favoritelang = ("Python", "Julia", "R")
      x1 = (1, 2, 3)
      x2 = (1, 2.0, "tmp")
      x3 = (a=2, b=1+2)

[1]: (a = 2, b = 3)
```

Рисунок 4.3: Определение кортежей

Продемонстрированы операции над кортежами: длина, индексация, обращение к полям именованного кортежа, проверка вхождения элемента через `in()` (рис. -Рисунок 4.4).

```
[2]: length(x2)
      x2[1], x2[2], x2[3]
      c = x1[2] + x1[3]
      x3.a, x3.b, x3[2]
      in("tmp", x2), 0 in x2

[2]: (true, false)
```

Рисунок 4.4: Операции над кортежами

4.2.2 Словари

Создан словарь `phonebook`, показаны `keys()`, `values()`, `pairs()`, `haskey()`, добавление и удаление элемента (`pop!`), объединение двух словарей

функцией `merge()` — при совпадении ключей побеждает значение из второго аргумента (рис. -Рисунок 4.5).

```
[3]: phonebook = Dict("Иванов И.И." => ("867-5309", "333-5544"),
                    "Бухгалтерия" => "555-2368")

keys(phonebook)
values(phonebook)
pairs(phonebook)
haskey(phonebook, "Иванов И.И.") # true
phonebook["Сидоров П.С."] = "555-3344"
pop!(phonebook, "Иванов И.И.") # ("867-5309", "333-5544"), ключ удалён

a = Dict("foo" => 0.0, "bar" => 42.0)
b = Dict("baz" => 17, "bar" => 13.0)
merge(a, b), merge(b, a)

[3]: (Dict{String, Real}{ "bar" => 13.0, "baz" => 17, "foo" => 0.0}, Dict{String, Real}{ "bar" => 42.0, "baz" => 17,
"foo" => 0.0})
```

Рисунок 4.5: Операции над словарями

4.2.3 Множества

Продемонстрированы создание множеств, проверка эквивалентности (`issetequal`), объединение, пересечение, разность множеств, проверка вложенности (`issubset`), добавление и удаление элемента (рис. -Рисунок 4.6).

```
[4]: A = Set([1, 3, 4, 5])
     B = Set("abracadabra") # уникальные буквы строки

S1 = Set([1,2]); S2 = Set([3,4])
issetequal(S1,S2) # false

S3 = Set([1,2,2,3,1,2,3,2,1]); S4 = Set([2,3,1])
issetequal(S3,S4) # true (оба {1,2,3})

C = union(S1,S2) # {1,2,3,4}
D = intersect(S1,S3) # {1,2}
E = setdiff(S3,S1) # {3}
issubset(S1,S4) # true
push!(S4, 99) # {1,2,3,99}
pop!(S4) # удаляет произвольный элемент (порядок не гарантирован)

[4]: 2
```

Рисунок 4.6: Операции над множествами

4.2.4 Массивы

Показаны создание пустых и типизированных массивов, векторов-строк и векторов-столбцов, многомерных массивов, массивов через генераторы (`list`

comprehension), функции `ones()`, `zeros()`, `fill()`, `repeat()`, `reshape()`, транспонирование, а также срезы и логическая индексация (`ar[0, 2, 4] == 14`, `findall()`) (рис. -Рисунок 4.7).

```
[5]: empty_array_1 = []
empty_array_2 = (Int64){}
empty_array_3 = (Float64){}
a = [1, 2, 3]
b = [1 2 3]
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]] # 3x3, каждый вектор – СТОЛБЕЦ
B = [[1 2 3]; [4 5 6]; [7 8 9]] # 3x3, каждый вектор – СТРОКА
c = rand(1,8)
C = rand(2,3)
D = rand(4,3,2) # трёхмерный, значения случайны каждый раз

roots = [sqrt(i) for i in 1:10]
ar_1 = [3*i^2 for i in 1:2:9] # [3, 27, 75, 147, 243]
ar_2 = [i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)] # [1, 9, 49, 81]

length(A), ndims(A), size(A), size(A,1)
copy(A)
ones(5); ones(2,3); zeros(4)
fill(3.5, (3,2))
repeat([1,2], 3, 3)
repeat([1 2], 3, 3)

a = collect(1:12)
b = reshape(a, (2,6))
b'
transpose(b)

ar = rand(10,20, 10, 5)
ar[:, 2]
ar[:, [2, 5]]
ar[:, 2:4]
ar[[2, 4, 6], [1, 5]]
ar[1, 3:end]
sort(ar, dims=1)
sort(ar, dims=2)
ar .> 14
findall(ar .> 14)
```

```
[5]: 26-element Vector{CartesianIndex{2}}:
 CartesianIndex(4, 1)
 CartesianIndex(5, 1)
 CartesianIndex(8, 1)
 CartesianIndex(9, 1)
 CartesianIndex(10, 1)
 CartesianIndex(1, 2)
 CartesianIndex(2, 2)
 CartesianIndex(3, 2)
 CartesianIndex(4, 2)
 CartesianIndex(7, 2)
 CartesianIndex(10, 2)
 CartesianIndex(1, 3)
 CartesianIndex(2, 3)
 CartesianIndex(3, 3)
 CartesianIndex(4, 3)
 CartesianIndex(5, 3)
 CartesianIndex(6, 3)
 CartesianIndex(7, 3)
 CartesianIndex(8, 3)
 CartesianIndex(9, 3)
 CartesianIndex(10, 3)
```

Would you like to get notified about Julia news?

Рисунок 4.7: Операции над массивами

4.3 Задания для самостоятельной работы (раздел 2.4)

4.3.1 Задание 1

В формулировке задания выражение $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$ содержит повторяющееся слагаемое $A \cap B$; исходя из структуры формулы (типичная форма разложения через дополнение) второе слагаемое интерпретировано как $A \cap \bar{B}$, где $\bar{B}$ – дополнение B до объединения всех трёх множеств. Результат $P = \{0, 1, 3, 4, 7, 9\}$ (рис. -Рисунок 4.8, верхняя часть).

4.3.2 Задание 2

Приведены собственные примеры операций над множествами со смешанными типами элементов (числа, строки, символы) – `union`, `intersect`, `setdiff` для множеств типа `Set {Any}` (рис. -Рисунок 4.8, нижняя часть).

```
[6]: A = Set([0, 3, 4, 9])
      B = Set([1, 3, 4, 7])
      C = Set([0, 1, 2, 4, 7, 8, 9])
      U = union(A, B, C)
      Bc = setdiff(U, B) # дополнение B

      P = union(intersect(A,B), intersect(A,Bc), intersect(A,C), intersect(B,C))
      println(P)

Set([0, 4, 7, 9, 3, 1])

[7]: S1 = Set([1, "два", 3.0, :четыре]) # смешанный тип Any
      S2 = Set(["два", 5, 3.0])
      union(S1, S2)
      intersect(S1, S2) # {"два", 3.0}
      setdiff(S1, S2)

[7]: Set{Any} with 2 elements:
      :четыре
      1
```

Рисунок 4.8: Задание 1 (множества A, B, C) и задание 2 (множества смешанных типов)

4.3.3 Задание 3

Выполнены все 14 пунктов задания: построение возрастающего, убывающего и «пирамидального» массивов (3.1–3.3), массивы на основе повторения элементов вектора `tmp` разными способами (3.4–3.8), массив степеней двойки с подсчётом вхождений цифры (3.9), вектор значений функции $y = e^x \cos(x)$ и его среднее (3.10), вектор пар индексов (3.11), вектор с элементами $2^i/i$ (3.12), вектор строковых меток (3.13) и обширный набор операций над случайными векторами x, y длины 250: разности, линейные комбинации, поэлементные тригонометрические выражения, сумма ряда, фильтрация по условию, статистика по чётности/кратности, сортировка одного вектора по перестановке другого, топ-10 значений и уникальные элементы (3.14) (рис. -Рисунок 4.9, -Рисунок 4.10).

```

[9]: N = 25
tmp = [4, 6, 3]

# 3.1-3.3
a31 = collect(1:N)
a32 = collect(N:-1:1)
a33 = vcat(1:N, N-1:-1:1)

# 3.4-3.8
a35 = fill(tmp[1], 10)
a36 = repeat(tmp, 10)
a37 = vcat(fill(tmp[1],11), fill(tmp[2],10), fill(tmp[3],10))
a38 = vcat(fill(tmp[1],10), fill(tmp[2],20), fill(tmp[3],30))

# 3.9
a39 = vcat(2^tmp[1], 2^tmp[2], fill(2^tmp[3], 4)) # [16, 64, 8, 8, 8, 8]
count_6 = sum(count(c -> c == '6', string(x)) for x in a39)
println(a39, " цифра 6 встречается: ", count_6) # ожидаемо: 2

# 3.10
using Statistics
x310 = 3:0.1:6
y310 = exp.(x310) .* cos.(x310)
mean(y310)

# 3.11
i311 = 3:3:36; j311 = 1:3:34
xs311 = 0.1 .* i311; ys311 = 0.2 .* j311
pairs311 = collect(zip(xs311, ys311))

# 3.12
v312 = [2.0^i / i for i in 1:25]

# 3.13
v313 = ["fn$i" for i in 1:30]

# 3.14
using Random
n = 250
x = rand(0:999, n)
y = rand(0:999, n)

d1 = y[2:end] .- x[1:end-1]
d2 = x[1:end-2] .+ 2 .* x[2:end-1] .- x[3:end]
d3 = sin.(y[1:end-1]) ./ cos.(x[2:end])
s = sum(exp.(-x[2:end]) ./ (x[1:end-1] .+ 10))

big_y = y[y .> 600]
idx_big = findall(y .> 600)
x_at_big_y = x[idx_big]

```

Рисунок 4.9: Задание 3, пункты 3.1–3.14 (код)

```

n = 250
x = rand(0:999, n)
y = rand(0:999, n)

d1 = y[2:end] .- x[1:end-1]
d2 = x[1:end-2] .+ 2 .* x[2:end-1] .- x[3:end]
d3 = sin.(y[1:end-1]) ./ cos.(x[2:end])
s = sum(exp.(-x[2:end]) ./ (x[1:end-1] .+ 10))

big_y = y[y .> 600]
idx_big = findall(y .> 600)
x_at_big_y = x[idx_big]

xbar = mean(x)
d4 = sqrt.(abs.(x .- xbar))

near_max = count(abs.(y .- maximum(y)) .<= 200)
n_even = count(iseven, x)
n_odd = count(isodd, x)
n_div7 = count(v -> v % 7 == 0, x)

x_sorted_by_y = x[sortperm(y)]
top10 = sort(x, rev=true)[1:10]
x_unique = unique(x)

[16, 64, 8, 8, 8, 8] цифра 6 встречается: 2
[9]: 227-element Vector{Int64}:
 545
 732
 268
 597
 294
 404
 637
 320
 379
 877
 418
 348
 360
  ⋮
 376
 445
 911
 767
 686
 586
 742
 934
 143
 162
 272
 702

```

Рисунок 4.10: Задание 3, пункты 3.1–3.14 (продолжение и результат)

При выполнении пункта 3.9 возникла синтаксическая ошибка в выражении `count(== '6', ...)` — оператор `==` нельзя использовать в такой позиции как унарный. Ошибка исправлена заменой на анонимную функцию `count(c -> c == '6', string(x))`.

4.3.4 Задание 4

Построен массив `squares` квадратов чисел от 1 до 100, последний элемент равен 10000 (рис. -Рисунок 4.11).

```
[10]: squares = [i^2 for i in 1:100]
      squares[end]
```

```
[10]: 10000
```

Рисунок 4.11: Задание 4: массив квадратов чисел

4.3.5 Задание 5

Подключён пакет `Primes`, сгенерирован массив первых 168 простых чисел, 89-е наименьшее простое число равно 461, срез с 89-го по 99-й элемент — [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523] (рис. -Рисунок 4.12).

```

[10]: squares = [i^2 for i in 1:100]
      squares[end]

[10]: 10000

[11]: import Pkg; Pkg.add("Primes")
      using Primes

      myprimes = primes(1000)
      length(myprimes)
      myprimes[89]
      myprimes[89:99]

      Updating registry at `~/julia/registries/General.toml`
      Resolving package versions...
      Updating `~/Project.toml`
      [27ebfcd6] + Primes v0.5.7
      Updating `~/Manifest.toml`
      [18e54dd8] + IntegerMathUtils v0.1.4
      [27ebfcd6] + Primes v0.5.7
      Precompiling project...
      ✓ LaTeXStrings
      ✓ Tables
      ✓ Crayons
      ✓ SentinelArrays
      ✓ StringManipulation
      ✓ PrettyTables
      ✓ DataFrames
      7 dependencies successfully precompiled in 51 seconds. 45 already precompiled.

[11]: 11-element Vector{Int64}:
      461
      463
      467
      479
      487
      491
      499
      503
      509
      521
      523

```

Рисунок 4.12: Задание 5: пакет Primes

4.3.6 Задание 6

Вычислены суммы: $\sum_{i=10}^{100} (i^3 + 4i^2) = 26852735$; $\sum_{i=1}^{25} \left(\frac{2^i}{i} + \frac{3^i}{i^2} \right) \approx 2.129 \times 10^9$;
сумма ряда с накопленными частичными суммами ≈ 170.167 (рис. -Рисунок 4.13).

```
[12]: # 6.1
s1 = sum(i^3 + 4i^2 for i in 10:100)
println(s1)  # ожидаемо: 26852735

# 6.2
s2 = sum(2.0^i/i + 3.0^i/i^2 for i in 1:25)
println(s2)  # порядок величины ~1.35 млрд (доминирует последнее слагаемое)

# 6.3
terms = [2k/(2k+1) for k in 1:19]  # 2/3, 4/5, 6/7, ..., 38/39
partial = cumsum(terms)
s3 = 1 + sum(partial)
println(s3)

26852735
2.1291704368143802e9
170.16669918752706
```

Рисунок 4.13: Задание 6: вычисление сумм

5 Результаты

- Все примеры раздела 2.2 (кортежи, словари, множества, массивы) воспроизведены, результаты совпадают с методическими указаниями.
- Задание 1 выполнено с учётом уточнения неоднозначной формулы (дополнение множества B).
- Задание 2 выполнено на собственных примерах множеств смешанных типов.
- Задание 3 выполнено полностью (все 14 пунктов), включая операции над случайными векторами длины 250.
- Задание 4 и 5 выполнены, значения простых чисел подтверждены (89-е простое число — 461).
- Задание 6 выполнено, все три суммы вычислены.

6 Обсуждение результатов

Работа с множествами и массивами в Julia подтвердила теоретические ожидания: операции над множествами (`union`, `intersect`,